Estructuras de datos

abstracción

se refiere al proceso mediante el que se representa en el mundo de las ideas un conjunto de objetos, seleccionando solo aquellas características que interesan, excluyendo toda propiedad que no importe.

Se debe tomar en cuenta que las características consideradas dependen de qué es importante para el observador.

Clases

conjunto de objetos reunidos bajo el mismo concepto

Una clase se define mediante:

- Un nombre que la identifica.
- Atributos de los objetos de la clase.
- Métodos asociados a los objetos de la clase.

Atributos

los atributos representan características de los objetos de la clase

métodos

los métodos representan comportamiento de los objetos de la clase

Según su función, los métodos se clasifican en

- Métodos constructores: son los responsables de crear nuevas instancias de la clase.
- Métodos analizadores: son los responsables de permitir consultas sobre los atributos de la clase.
- Métodos modificadores: son los responsables de permitir modificaciones sobre los atributos de la clase.

Objetos

Un objeto es una instancia o particularización de una clase

los términos instancia y objeto son intercambiables

Las propiedades de cada objeto son definidas mediante los valores que toman sus atributos.

Arreglos

es una secuencia de elementos del mismo tipo que son almacenados de forma contigua

Análisis de algoritmos

la ejecución de todo algoritmo demanda recursos computacionales

la complejidad de un algoritmo representa la cantidad del recurso que el programa demanda para su ejecución

la complejidad temporal mide el tiempo empleado y la complejidad espacial mide el espacio requerido.

Es mucho más frecuente el análisis de tiempo que el de espacio, por ello, cuando se encuentren ejemplos en textos y otras fuentes, se debería asumir que se habla de complejidad temporal, a menos que se indique lo contrario.

Como es más costoso el recurso de tiempo que el de espacio, resulta más valioso realizar el cálculo de la complejidad temporal,

complejidad espacial

Mide la demanda de la memoria RAM

Para calcular complejidades espaciales de forma teórica, se sugieren los siguientes pasos:

A. Identificar las variables que describen el tamaño del problema a resolver. Por ejemplo, si se están sumando dos matrices de dimensiones n x m, el tamaño del problema estaría descrito por la variable n (el alto de las matrices) y por la variable m (el ancho de las matrices).

B. Encuentre una función que relacione el tamaño del problema con el número de variables declaradas en el programa. Esta función refleja la complejidad espacial porque entre más variables se declaren, más espacio en memoria RAM requeriría el algoritmo para ejecutarse.

complejidad temporal

Mide la demanda del procesador

Procedimiento para calcular complejidad temporal de forma teórica

A. Identificar las variables que describen el tamaño del problema a resolver. Por ejemplo, si se están sumando dos matrices de dimensiones n x m, el tamaño del problema estaría descrito por la variable n (el alto de las matrices) y por la variable m (el ancho de las matrices).

B. Definir las operaciones básicas que desea contabilizar. En el ejemplo anterior de suma de matrices, importaría contar cuántas sumas de celdas se realizan.

C. Encontrar una función que relacione el tamaño del problema con el número de operaciones básicas ejecutadas, analizando el peor escenario posible. Esta función refleja la complejidad temporal, porque entre más operaciones se ejecuten, más tiempo se demoraría el algoritmo en ejecutar.

Medidas de eficiencia de algoritmos

Notación asintótica O grande

La notación asintótica es una forma de representación que permite describir la eficiencia de un algoritmo en función de su tasa de crecimiento, es decir, proyecta el comportamiento del algoritmo conforme al aumento del tamaño de sus entradas.

se centra en un análisis del peor caso es decir, en el escenario más adverso imaginable, que haga que la función ejecute el máximo número de operaciones posible.

Puede usarse para describir tanto complejidad temporal como espacial.

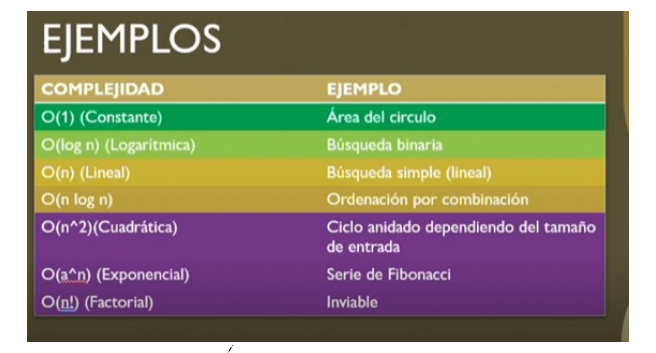
Dada una función f(n) se dice que un algoritmo tiene complejidad temporal O(f(n)) si la cantidad de operaciones que ejecuta siempre está por debajo de un múltiplo constante de f(n), para un valor grande de n.

Reglas para el cálculo de complejidad

A. Para calcular la complejidad temporal de una secuencia de instrucciones en Java, Se debe hallar la complejidad de cada instrucción por separado y tomar la más grande de estas.

B. Para calcular la complejidad temporal de una asignación en Java, hay que hallar la complejidad del cálculo de la expresión que está siendo asignada.

C. Para calcular la complejidad temporal de un condicional en Java, hay que hallar la complejidad de evaluar la condición de cada instrucción por separado y tomar la más grande de estas.



EJEMPLOS

COMPLEJIDAD	EJEMPLO
O(1) (Constante)	Área del circulo
O(log n) (Logarítmica)	Búsqueda binaria
O(n) (Lineal)	Búsqueda simple (lineal)
O(n log n)	Ordenación por combinación
O(n^2)(Cuadrática)	Ciclo anidado dependiendo del tamaño de entrada
O(a^n) (Exponencial)	Serie de Fibonacci
O(n!) (Factorial)	Inviable

Principales órdenes de complejidad

Ordenes de complejidad mas comunes

Orden constante O(1)	Representa la complejidad de un algoritmo que se ejecutará siempre en el mismo tiempo o espacio, sin importar el tamaño de las entradas.
Orden logarítmico O(log n)	Representa la complejidad de un algoritmo cuyo tiempo de ejecución se incrementará más lentamente a medida que el tamaño del conjunto de entradas aumente.
Orden lineal O(n)	Representa la complejidad de un algoritmo que se ejecutará en un tiempo que crecerá linealmente, en proporción directa con el tamaño del conjunto de entradas.
Orden n log de n O(n log n)	Representa la complejidad de un algoritmo que se ejecutará en un tiempo que crecerá de manera aproximada a la lineal, pero, multiplicado por un factor que aumenta cada vez más lentamente a medida que se incrementa el tamaño del conjunto de entradas.
Orden cuadrático O(n2)	Representa la complejidad de un algoritmo que se ejecutará en un tiempo que crecerá en proporción directa con el cuadrado del tamaño del conjunto de entradas.
Orden exponencial O(an)	Representa la complejidad de un algoritmo que se ejecutará en un tiempo que crecerá en un factor de a con cada adición al conjunto de datos de entrada. Proporción directa con el cuadrado del tamaño del conjunto de entradas.

Otras notaciones asintóticas diferentes a la O grande

Notación asintótica Ω grande	se centra en analizar el mejor caso Dada una función f(n) se dice que un algoritmo tiene complejidad temporal Ω (f(n)) si la cantidad de operaciones que ejecuta siempre está por encima de un múltiplo constante de f(n), para un valor grande de n
Notación asintótica Θ grande	La notación asintótica Θ grande (la letra griega Theta) se usa para describir la tasa de crecimiento en el mejor y el peor caso del tiempo de ejecución, o el espacio de memoria empleado, de un algoritmo cuando la tasa es la misma. Dada una función f(n) se dice que un algoritmo tiene complejidad temporal Θ(f(n)) si y solo si el orden de complejidad en el mejor escenario es Ω (f(n)) y el orden de complejidad en el peor escenario es O(f(n)).

java.util.*;

cuando importas la librería java.util.*; en un programa en Java, tienes acceso a varias clases y estructuras de datos incluidas en el paquete java.util. Entre las clases disponibles en este paquete se encuentran :

ArrayList: Una implementación de lista basada en un array redimensionable.

LinkedList: Una implementación de lista doblemente enlazada que también implementa las interfaces List, Deque y Queue.

Queue: Una interfaz que define el comportamiento de una cola, que puede ser implementada por varias clases, incluyendo LinkedList.

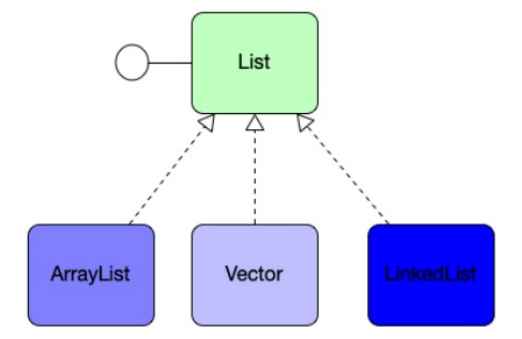
Stack: Una clase que implementa una pila (LIFO - Last In, First Out).

Vector: Una estructura de datos similar a ArrayList, pero sincronizada y menos utilizada en código moderno.

Listas

Una lista es una secuencia finita de elementos del mismo tipo.

Recordemos que el interface List esta implementado por varias clases concretamente por Vector, LinkedList y ArrayList



En Java, para utilizar las clases ArrayList, LinkedList y Vector, necesitas importar las siguientes clases de la biblioteca estándar de Java:

ArrayList	import java.util.ArrayList;
LinkedList	import java.util.LinkedList;
Vector	import java.util.Vector;
todas	import java.util.*;

Listas en lenguaje Java

En el área de la programación orientada a objetos, una interfaz es un conjunto de métodos definidos sin implementación.

Implementaciones suministradas por Java para la interfaz List<E>

dos implementaciones de Java para la interfaz List<E>:

El tipo E es genérico, es decir, puede ser cualquier clase en Java.

List<E>	es una interfaz de Java que representa una lista de elementos de tipo E
List<Integer>	lista de números enteros,
List<Double>	lista de números flotantes
List<String>	lista de cadenas de texto
List<Persona>	lista de personas,

Conceptos básicos

Posición	Posicion del elemento especifico
Tamaño	cantidad de elementos
Ordinales	El primer elemento de la lista es el que se encuentra en la posición 0, el segundo elemento es el que se encuentra en la posición 1, El último elemento de la lista es el que se encuentra en la posición n-1,
Siguiente	Cada elemento de la lista, excepto el último, tiene un siguiente elemento.
Anterior	Cada elemento de la lista, excepto el primero, tiene un anterior elemento.
Recorridos	Un recorrido de una lista es una forma de visitar todos los elementos de la lista.
Igualdad	Dos listas son iguales si y solo si tienen el mismo tamaño y almacenan los mismos elementos en las mismas posiciones.
Contenencia	Una lista está contenida dentro de otra si y solo si todo elemento de la primera está presente dentro de la segunda, así sea en distinto orden.
Sub-listas	Una lista es sub-lista de otra si y solo si todos los elementos de la primera aparecen en el mismo orden y consecutivos sin otros elementos que esten contenidos o no dentro de la primera lista a partir de alguna posición de la segunda lista.

Recorridos:

- Recorrido al derecho, — comenzando por el primer elemento y terminando en el último.
- Recorrido al revés, — comenzando por el último elemento y terminando en el primero

Operaciones básicas

A Consulta, que sirve para obtener el valor presente en una posición dada.

B.Modificación, para cambiar el valor presente en una posición dada.

C. Inserción, para añadir un nuevo elemento en cierta posición de la lista.

D. Eliminación, para quitar de la lista el elemento que está en cierta posición

La clase Vector

Import — importar import java.util.*;

creamos un vector u objeto de la clase Vector — **Vector vector = new Vector(20, 5);** — Tenemos un vector con una dimensión inicial de 20 elementos. Si rebasamos dicha dimensión y guardamos 21 elementos la dimensión del vector crece a 25.

Al segundo constructor, solamente se le pasa la dimensión inicial. — **Vector vector = new Vector(20);** — Si se rebasa la dimensión inicial guardando 21 elementos, la dimensión del vector se duplica

Añadir elementos al vector

- **v.addElement("uno");**
- **v.insertElementAt("tres", 2);**

Eliminar elementos de un vector

- eliminar todos los elementos de un vector — removeAllElements
- eliminar un elemento concreto — v.removeElement("elemento"); — v.removeElementAt("indice o posicion");

Acceso a los elementos de un vector — elementAt.

```java
for(int i=0; i<v.size(); i++){
        System.out.print(v.elementAt(i)+"\t");
    }
```

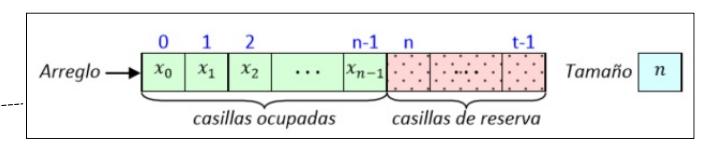
Diferencia entre ArrayList y Vector In java

Cambiar el tamaño: — ArrayList crece a la mitad de su tamaño cuando se redimensiona, mientras que Vector dobla su tamaño por defecto cuando crece.

Sincronización: — ArrayList no está sincronizado, lo que significa que múltiples hilos pueden trabajar en ArrayList al mismo tiempo.

ArrayList<E>

Implementa listas a través de vectores (arreglos dinámicos de tamaño variable)



Implementacion

Importar — import java.util.ArrayList;

sintaxis general — **ArrayList<Type> arrayList= new ArrayList<>();** — type representa el tipo de datos respectivo de Java ArrayList

- **ArrayList<String> arrayList= new ArrayList<>();**
- **ArrayList<Integer> arrayList= new ArrayList<>();**

```java
import java.util.ArrayList;
class Main {
   public static void main(String[] args){
      ArrayList<String> colors = new ArrayList<>();
      colors.add("blue");
      colors.add("red");
      colors.add("green");
      System.out.println("ArrayList: " + colors);
   }
}
```

Acceso a los elementos — . get ( )

```java
import java.util.ArrayList;
class Main {
   public static void main(String[] args) {
      ArrayList<String> clothes = new ArrayList<>();
      clothes.add("jacket");
      clothes.add("shirt");
      clothes.add("pullover");
      System.out.println("ArrayList: " + clothes);
      String str = clothes.get(2);
      System.out.print("Element at index 2: " + str);
   }
}
```

Modificar elementos — . set ( )

```java
import java.util.ArrayList;
class Main {
   public static void main(String[] args) {
      ArrayList<String> colors = new ArrayList<>();
      colors.add("blue");
      colors.add("red");
      colors.add("green");
      System.out.println("ArrayList: " + colors);
      colors.set(2, "yellow");
      System.out.println("Modified ArrayList: " + colors);
   }
}
```

Añadir elementos — . add ( )

```java
import java.util.ArrayList;
class Main {
   public static void main(String[] args){
      ArrayList<String> colors = new ArrayList<>();
      colors.add("blue");
      colors.add("red");
      colors.add("green");
      System.out.println("ArrayList: " + colors);
   }
}
```

Eliminar elementos — . remove( )

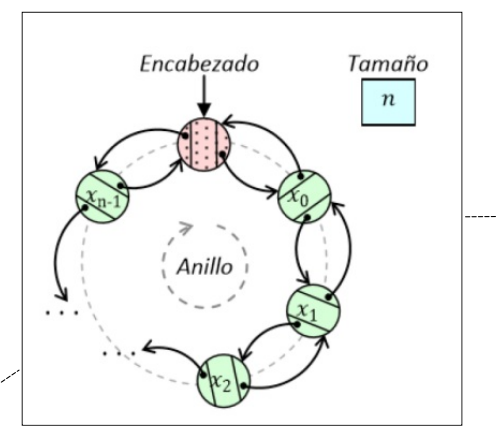
```java
import java.util.ArrayList;
class Main {
   public static void main(String[] args) {
      ArrayList<String> colors = new ArrayList<>();
      colors.add("blue");
      colors.add("red");
      colors.add("green");
      System.out.println("ArrayList: " + colors);
      String color = colors.remove(1);
      System.out.println("ArrayList: " + colors);
      System.out.println("Removed Element: " + color);
   }
}
```

Ejemplo

```java
import java.util.ArrayList;

public class EjemploArrayList {
   public static void main(String[] args) {
      // Crear un ArrayList de Strings
      ArrayList<String> lista = new ArrayList<>();

      // Agregar elementos
      lista.add("Manzana");
      lista.add("Banana");
      lista.add("Naranja");

      // Acceder y mostrar un elemento
      System.out.println("Elemento en índice 0: " + lista.get(0));

      // Modificar un elemento
      lista.set(1, "Fresa");
      System.out.println("Lista después de modificar: " + lista);

      // Eliminar un elemento
      lista.remove("Naranja");
      System.out.println("Lista después de eliminar 'Naranja': " + lista);

      // Verificar si contiene un elemento
      boolean contiene = lista.contains("Manzana");
      System.out.println("¿Contiene 'Manzana'? " + contiene);

      // Iterar sobre los elementos
      System.out.println("Elementos en la lista:");
      for (String fruta : lista) {
         System.out.println(fruta);
      }

      // Vaciar la lista
      lista.clear();
      System.out.println("Tamaño de la lista después de clear: " + lista.size());
   }
}
```

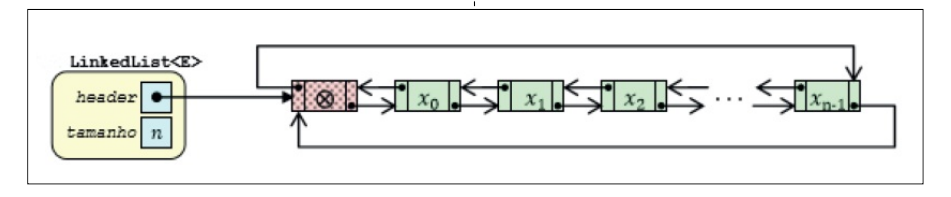
LinkedList<E>

implementa listas a través de nodos doblemente encadenados en anillo con encabezado.



la clase LinkedList es parte del paquete java.util y representa una lista enlazada doblemente, lo que significa que cada elemento (o nodo) de la lista está vinculado tanto al siguiente como al anterior nodo.

LinkedList puede utilizarse como una lista, una cola o una pila, y es particularmente útil cuando necesitas insertar o eliminar elementos frecuentemente, ya que estas operaciones son más eficientes que en un ArrayList para listas largas



Implementacion

importar — import java.util.LinkedList;

Crear un LinkedList

```java
LinkedList<String> lista = new LinkedList<>();  // Lista para almacenar Strings
LinkedList<Integer> numeros = new LinkedList<>();  // Lista para almacenar enteros
```

Agregar Elementos

```java
lista.add("Manzana");      // Agrega al final
lista.addFirst("Banana");  // Agrega al inicio
lista.addLast("Naranja");  // Agrega al final
lista.add(1, "Fresa");     // Inserta en el índice 1
```

Acceder a Elementos

```java
String primerElemento = lista.getFirst();  // Obtiene el primer elemento ("Banana")
String elemento = lista.get(1);            // Obtiene el elemento en el índice 1 ("Fresa")
System.out.println(primerElemento);        // Imprime "Banana"
```

Modificar Elementos

```java
lista.set(2, "Kiwi");  // Cambia el elemento en el índice 2 ("Manzana") por "Kiwi"
```

Eliminar Elementos

```java
lista.remove(1);          // Elimina el elemento en el índice 1 ("Fresa")
lista.remove("Kiwi");     // Elimina "Kiwi"
lista.removeFirst();      // Elimina el primer elemento ("Banana")
lista.removeLast();       // Elimina el último elemento ("Naranja")
```

Obtener el Tamaño del LinkedList

```java
int tamaño = lista.size();  // Devuelve el tamaño de la lista
System.out.println("Tamaño de la lista: " + tamaño);
```

Iterar sobre un LinkedList

Con un bucle for:

```java
for (int i = 0; i < lista.size(); i++) {
    System.out.println(lista.get(i));
}
```

Con un bucle for-each:

```java
for (String elemento : lista) {
    System.out.println(elemento);
}
```

Con un Iterator:

```java
Iterator<String> iterator =
lista.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

Verificar si un Elemento Existe

```java
boolean contiene = lista.contains("Manzana");  // Devuelve true si "Manzana" está en la lista
System.out.println("¿Contiene 'Manzana'? " + contiene);
```

Vaciar el LinkedList

```java
lista.clear();  // Elimina todos los elementos de la lista
System.out.println("Tamaño después de clear: " + lista.size());  // Debería mostrar 0
```

Ejemplo

```java
import java.util.LinkedList;
import java.util.Iterator;

public class EjemploLinkedList {
    public static void main(String[] args) {
        // Crear un LinkedList de Strings
        LinkedList<String> lista = new LinkedList<>();

        // Agregar elementos
        lista.add("Manzana");
        lista.addFirst("Banana");
        lista.addLast("Naranja");
        lista.add(1, "Fresa");

        // Acceder y mostrar elementos
        System.out.println("Primer elemento: " + lista.getFirst());
        System.out.println("Elemento en índice 1: " + lista.get(1));

        // Modificar un elemento
        lista.set(2, "Kiwi");
        System.out.println("Lista después de modificar: " + lista);

        // Eliminar elementos
        lista.remove(1);
        lista.remove("Kiwi");
        lista.removeFirst();
        lista.removeLast();
        System.out.println("Lista después de eliminar elementos: " + lista);

        // Verificar si contiene un elemento
        boolean contiene = lista.contains("Manzana");
        System.out.println("¿Contiene 'Manzana'? " + contiene);

        // Iterar sobre los elementos
        lista.add("Pera");
        lista.add("Uva");
        System.out.println("Elementos en la lista:");
        for (String fruta : lista) {
            System.out.println(fruta);
        }

        // Usar un Iterator
        Iterator<String> iterator = lista.iterator();
        System.out.println("Elementos usando Iterator:");
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }

        // Vaciar la lista
        lista.clear();
        System.out.println("Tamaño de la lista después de clear: " + lista.size());
    }
}
```

Ventajas de uso de LinkedList o de ArrayList

En una LinkedList, cuando insertas o eliminas un elemento, solo necesitas cambiar los punteros del nodo anterior y posterior al nuevo nodo. Esto tiene una complejidad O(1) para inserciones y eliminaciones.

LinkedList es más adecuada si necesitas realizar **inserciones y eliminaciones frecuentes** en cualquier parte de la lista, especialmente en el medio

En una ArrayList, cuando insertas o eliminas un elemento en el medio de la lista, todos los elementos que vienen después tienen que moverse para hacer espacio o para llenar el hueco. Esto tiene una complejidad O(n) para inserciones y eliminaciones en el medio.

ArrayList es mejor si necesitas **acceder rápidamente** a elementos por su índice y las inserciones o eliminaciones se realizan al final de la lista, ya que tiene un acceso más rápido a los elementos debido a que se almacena en un array continuo en la memoria

LinkedList es más eficiente que **ArrayList** para **inserciones y eliminaciones** en el medio de la lista,

```java
import java.util.ArrayList;
import java.util.LinkedList;

public class ListPerformanceComparison {

    public static void main(String[ ] args) {

        // Tamaño inicial de la lista
        int size = 100000;

        // Creamos una LinkedList y una ArrayList con el mismo contenido inicial
        LinkedList<Integer> linkedList = new LinkedList<>();
        ArrayList<Integer> arrayList = new ArrayList<>();

        // Llenar ambas listas con valores iniciales
        for (int i = 0; i < size; i++) {
            linkedList.add(i);
            arrayList.add(i);
        }

        // Medir tiempo de inserción en la mitad de la LinkedList
        long startTimeLinkedList = System.currentTimeMillis();
        for (int i = 0; i < 1000; i++) {
            linkedList.add(size / 2, -1);  // Insertar en la mitad
        }
        long endTimeLinkedList = System.currentTimeMillis();
        System.out.println("Tiempo de inserción en la mitad para LinkedList: "
                + (endTimeLinkedList - startTimeLinkedList) + " ms");

        // Medir tiempo de inserción en la mitad de la ArrayList
        long startTimeArrayList = System.currentTimeMillis();
        for (int i = 0; i < 1000; i++) {
            arrayList.add(size / 2, -1);  // Insertar en la mitad
        }
        long endTimeArrayList = System.currentTimeMillis();
        System.out.println("Tiempo de inserción en la mitad para ArrayList: "
                + (endTimeArrayList - startTimeArrayList) + " ms");

        // Medir tiempo de eliminación en la mitad de la LinkedList
        long startTimeRemoveLinkedList = System.currentTimeMillis();
        for (int i = 0; i < 1000; i++) {
            linkedList.remove(size / 2);  // Eliminar en la mitad
        }
        long endTimeRemoveLinkedList = System.currentTimeMillis();
        System.out.println("Tiempo de eliminación en la mitad para LinkedList: "
                + (endTimeRemoveLinkedList - startTimeRemoveLinkedList) + "
ms");

        // Medir tiempo de eliminación en la mitad de la ArrayList
        long startTimeRemoveArrayList = System.currentTimeMillis();
        for (int i = 0; i < 1000; i++) {
            arrayList.remove(size / 2);  // Eliminar en la mitad
        }
        long endTimeRemoveArrayList = System.currentTimeMillis();
        System.out.println("Tiempo de eliminación en la mitad para ArrayList: "
                + (endTimeRemoveArrayList - startTimeRemoveArrayList) + " ms");
    }
}
```

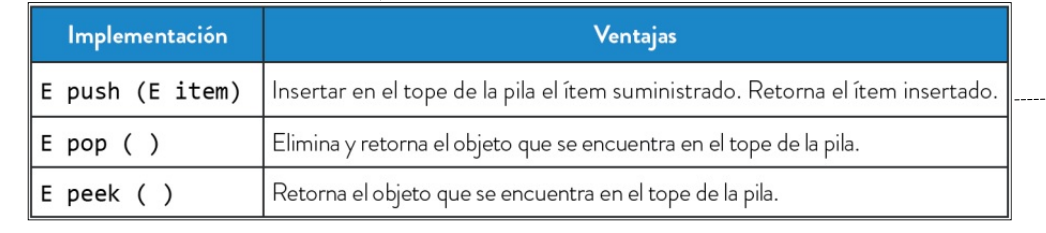
Salida

```
Tiempo de inserción en la mitad para LinkedList: 5 ms
Tiempo de inserción en la mitad para ArrayList: 300 ms
Tiempo de eliminación en la mitad para LinkedList: 4 ms
Tiempo de eliminación en la mitad para ArrayList: 280 ms
```

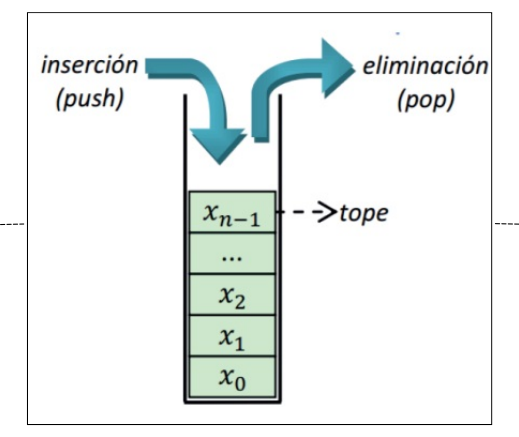
Pilas

es una estructura de datos lineal, que solo permite inserciones y eliminaciones en uno de los extremos, llamado tope.

La clase Stack<E> de Java, que extiende de la clase Vector<E>,



Implementación	Ventajas
E push (E item)	Insertar en el tope de la pila el ítem suministrado. Retorna el ítem insertado.
E pop ( )	Elimina y retorna el objeto que se encuentra en el tope de la pila.
E peek ( )	Retorna el objeto que se encuentra en el tope de la pila.



```java
import java.util.Stack;

public class PilaEjemplo {

    public static void main(String[] args) {

        // Crear una pila
        Stack<String> pila = new Stack<>();

        // Agregar elementos a la pila
        pila.push("Elemento 1");
        pila.push("Elemento 2");
        pila.push("Elemento 3");

        // Mostrar el elemento en la cima (pero no eliminarlo)
        System.out.println("Elemento en la cima de la pila: " + pila.peek());

        // Quitar elementos de la pila (LIFO)
        System.out.println("Desapilando: " + pila.pop());
        System.out.println("Desapilando: " + pila.pop());

        // Mostrar el elemento restante en la cima
        System.out.println("Elemento en la cima después de quitar dos elementos: " +
pila.peek());

        // Ver si la pila está vacía
        System.out.println("¿La pila está vacía? " + pila.empty());
    }
}
```

Ejemplos de uso de pilas en la vida real:

1. Navegador Web (Historial de Navegación):
Los navegadores web utilizan pilas para manejar el historial de páginas que has visitado. Cada vez que visitas una nueva página, esa página se apila sobre las anteriores. Si haces clic en "Atrás", el navegador desapila la página actual y te lleva a la anterior. Si luego haces clic en "Adelante", se vuelve a apilar la página que se había eliminado.
Acción "Atrás": Desapila la última página visitada.
Acción "Adelante": Apila nuevamente la página eliminada si no se ha visitado otra nueva.

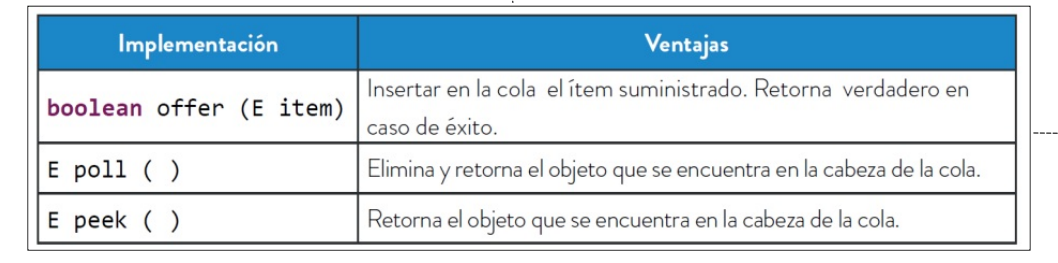
2. Deshacer/Redo en editores de texto:
La funcionalidad de "deshacer" (Ctrl+Z) y "rehacer" (Ctrl+Y) en los editores de texto también usa pilas. Cada vez que realizas una acción (como escribir una palabra), esta se apila en una estructura de pila. Cuando haces "deshacer", se desapila la última acción y cuando haces "rehacer", se vuelve a apilar la acción si no has realizado ningún cambio nuevo.
Acción "Deshacer": Elimina el último cambio realizado (desapilar).
Acción "Rehacer": Restaura el último cambio eliminado (volver a apilar).

3. Recursividad:
En programación, la recursividad utiliza pilas para almacenar el estado de cada llamada recursiva. Cada vez que una función se llama a sí misma, su estado actual se apila. Una vez que se alcanza la condición base (caso donde se deja de hacer recursión), las funciones se desapilan en orden inverso hasta que la pila queda vacía.
Ejemplo: En algoritmos de búsqueda o problemas de cálculo factorial, las funciones recursivas apilan las llamadas y luego las resuelven en orden inverso cuando la recursión termina.

Queue

Colas

es una estructura de datos lineal, que permite inserciones en un extremo (llamado cola) y eliminaciones del otro extremo (llamado cabeza).



Implementación	Ventajas
boolean offer (E item)	Insertar en la cola el ítem suministrado. Retorna verdadero en caso de éxito.
E poll ( )	Elimina y retorna el objeto que se encuentra en la cabeza de la cola.
E peek ( )	Retorna el objeto que se encuentra en la cabeza de la cola.

```java
import java.util.LinkedList;
import java.util.Queue;

public class ColaEjemplo {

    public static void main(String[] args) {

        // Crear una cola
        Queue<String> cola = new LinkedList<>();

        // Agregar elementos a la cola
        cola.add("Elemento 1");
        cola.add("Elemento 2");
        cola.add("Elemento 3");

        // Mostrar el primer elemento en la cola (pero no eliminarlo)
        System.out.println("Elemento en la cabeza de la cola: " + cola.peek());

        // Eliminar elementos de la cola (FIFO)
        System.out.println("Desencolando: " + cola.poll());
        System.out.println("Desencolando: " + cola.poll());

        // Mostrar el elemento restante en la cabeza de la cola
        System.out.println("Elemento en la cabeza después de desencolar dos elementos: " + cola.peek());

        // Ver si la cola está vacía
        System.out.println("¿La cola está vacía? " + cola.isEmpty());
    }
}
```

Diferencias entre List<E> y Arreglos sencillos

Arreglos — List<E>

Tamaño

Arreglos: El tamaño de un arreglo es **fijo** en el momento de su creación

Las estructuras que implementan la interfaz List<E> (como ArrayList, LinkedList) son **dinámicas**. Esto significa que su tamaño puede cambiar (crecer o reducirse) automáticamente al agregar o eliminar elemento

```java
// Arreglo (tamaño fijo)
int[ ] arreglo = new int[5];  // Solo tiene espacio para 5 elementos

// List (tamaño dinámico)
List<Integer> lista = new ArrayList<>();  // El tamaño inicial puede ser cero y crece dinámicamente
lista.add(10);  // Agregas elementos según sea necesario
```

Flexibilidad

Arreglos: Los arreglos solo pueden almacenar elementos de un **tipo específico**. Por ejemplo, un arreglo de enteros (int[ ]) solo puede contener enteros

List<E>: Las estructuras basadas en List<E> son **genéricas**, lo que significa que puedes especificar el tipo de elemento que quieres almacenar mediante la notación de genéricos (<E>). Además, son más flexibles para agregar, eliminar o acceder a los elementos.

```java
// Arreglo de enteros
int[ ] numeros = {1, 2, 3, 4, 5};

// List de enteros
List<Integer> numerosLista = new ArrayList<>();
numerosLista.add(1);
numerosLista.add(2);
```

Métodos y funcionalidad

Arreglos: Los arreglos tienen una funcionalidad básica. Solo te permiten acceder a elementos por su índice, establecer valores y obtener el tamaño. Además, los arreglos no tienen métodos útiles como búsqueda, eliminación, o agregar elementos de manera eficiente

List<E>: Las estructuras que implementan List<E> tienen muchos **métodos adicionales** para agregar (add), eliminar (remove), buscar (indexOf), y manipular los elementos de manera más fácil y eficiente. Además, proporcionan métodos de ordenación (sort), sublistas (subList), e iteración simplificada.

```java
// Acceso en un arreglo
int[ ] arreglo = {10, 20, 30};
System.out.println(arreglo[0]);  // Acceso por índice en arreglo

// Métodos útiles en una lista
List<Integer> lista = new ArrayList<>();
lista.add(10);        // Agregar un elemento
lista.remove(0);      // Eliminar un elemento
lista.indexOf(10);    // Buscar un elemento
```

Tipo de datos primitivos

Arreglos: Los arreglos pueden almacenar **tipos primitivos** como int, char, double, etc., lo cual es útil cuando deseas trabajar con esos tipos directamente.

List<E>: Las estructuras de List<E> no pueden almacenar directamente tipos primitivos; en su lugar, deben usar las **clases envolventes** (como Integer en lugar de int, Double en lugar de double, etc.).

```java
// Arreglo de tipos primitivos
int[ ] numeros = {1, 2, 3};

// Lista de tipos envolventes
List<Integer> listaNumeros = new ArrayList<>();
listaNumeros.add(1);  // Integer, no int
```

Eficiencia de inserción y eliminación

Arreglos: En un arreglo, agregar o eliminar elementos en posiciones específicas puede ser **ineficiente**, ya que requiere mover manualmente los elementos posteriores para mantener el orden del arreglo. Los arreglos son más eficientes cuando solo necesitas acceder a elementos por su índice.

List<E>: Las listas como ArrayList permiten insertar y eliminar elementos de manera más fácil y tienen métodos dedicados para hacerlo. Sin embargo, la eficiencia de las operaciones varía dependiendo de la implementación.

Por ejemplo:
En un **ArrayList**, agregar elementos al final es muy eficiente (amortizado O(1)), pero agregar o eliminar elementos al principio o en el medio es menos eficiente (O(n)).

En una **LinkedList**, agregar o eliminar elementos en cualquier posición es eficiente (O(1) en inserciones), pero el acceso por índice es más lento (O(n)).

```java
// En ArrayList, agregar al final es eficiente
List<Integer> lista = new ArrayList<>( );
lista.add(10);  // O(1) - agregar al final
lista.add(0, 5);  // O(n) - insertar en el medio requiere
desplazamiento
```

Iteración

Arreglos: Puedes iterar a través de los elementos de un arreglo usando bucles for o for-each

List<E>: Además de los bucles tradicionales, las listas ofrecen métodos como forEach, y son compatibles con iteradores, lo que facilita recorrer los elementos de manera más flexible.

```java
// Iterando un arreglo
int[] arreglo = {1, 2, 3};
for (int i = 0; i < arreglo.length; i++) {
    System.out.println(arreglo[i]);
}

// Iterando una lista
List<Integer> lista = new ArrayList<>();
lista.add(1);
lista.add(2);
lista.add(3);
for (int num : lista) {
    System.out.println(num);
}
```

Resumen de diferencias clave:

Tamaño
Arreglos -> Fijo
List<E> -> Dinámico (se ajusta automáticamente)

Almacenamiento de tipos
Arreglos -> Puede almacenar tipos primitivos
List<E> -> Usa clases envolventes para tipos primitivos

Acceso
Arreglos -> Acceso directo por índice
List<E> -> Acceso por índice en ArrayList, pero más lento en LinkedList

Métodos disponibles
Arreglos -> Limitados
List<E> -> Muchos métodos útiles (add, remove, sort, etc.)

Eficiencia de inserción/eliminación
Arreglos -> Ineficiente para inserciones/eliminaciones en el medio
List<E> -> Más flexible, pero varía según la implementación

Uso de genéricos
Arreglos -> No
List<E> -> Sí, usa genéricos (List<E>)